

入門指南

本指南協助您在專案中設定 SpreadsheetExporter，並建立第一個 Excel 匯出功能。

安裝

安裝核心函式庫：

```
dotnet add package CloudyWing.SpreadsheetExporter
```

安裝 Excel 實作套件 (ClosedXML)：

```
dotnet add package CloudyWing.SpreadsheetExporter.Renderer.ClosedXML
```

設定

SpreadsheetExporter 目前有兩種啟動方式：

- 非 DI 專案：使用 `SpreadsheetManager.SetRenderer(...)`，適合 Console、腳本、單機工具與快速驗證。
- 已使用 DI 的專案：在組合根註冊 `ISpreadsheetRenderer`，再由應用程式流程建立 `SpreadsheetDocument`。

非 DI 專案

在應用程式啟動時 (例如 `Program.cs`) 登錄 renderer：

```
using CloudyWing.SpreadsheetExporter;  
using CloudyWing.SpreadsheetExporter.Renderer.ClosedXML;
```

```
SpreadsheetManager.SetRenderer(static () => new ExcelRenderer());
```

⊗ IMPORTANT

請務必透過 `SpreadsheetManager.CreateDocument()` 建立文件實例，而非直接使用 `new SpreadsheetDocument(new ExcelRenderer())`。統一透過 `SpreadsheetManager` 管理，可確保全域設定 (如 `DefaultCellStyles`) 正確套用。

DI 專案

若你的應用程式本來就使用 `Microsoft.Extensions.DependencyInjection`，可以直接在組合根註冊 `renderer`：

```
using CloudyWing.SpreadsheetExporter;
using CloudyWing.SpreadsheetExporter.Renderer.ClosedXML;
using Microsoft.Extensions.DependencyInjection;

ServiceCollection services = new();
services.AddSingleton<ISpreadsheetRenderer>(_ => new ExcelRenderer());

ServiceProvider serviceProvider = services.BuildServiceProvider();
ISpreadsheetRenderer renderer = serviceProvider.GetRequiredService<ISpreadsheetRenderer>();

SpreadsheetDocument document = new SpreadsheetDocument(renderer);
```

若需要初始化預設值，也可以在組合根先設定：

```
SpreadsheetManager.DefaultCellStyles = new CellStyleConfiguration {
    HeaderStyle = SpreadsheetManager.DefaultCellStyles.HeaderStyle with {
        HasBorder = true
    }
};
```

自訂 `renderer` 的生命週期選擇

DI 專案中，`ISpreadsheetRenderer` 的生命週期取決於自訂 `renderer` 是否保存狀態，以及是否依賴其他 `scoped` 服務。

- `Singleton`：適合無狀態 `renderer`。`renderer` 不保存跨呼叫狀態，也不依賴 `scoped` 服務時，可優先使用。
- `Scoped`：適合 `renderer` 需要使用目前 `request`、租戶、語系或其他 `scoped` 資訊。
- `Transient`：適合 `renderer` 內部會累積暫存狀態，或包裝的物件不適合重用。

例如下列情境應考慮 `Scoped` 或 `Transient`，不要直接使用 `Singleton`：

- `renderer` 依賴 `scoped` 的 `tenant provider`、使用者資訊或資料存取服務。
- `renderer` 內部持有可變欄位，且每次匯出都會改寫。
- `renderer` 包裝的第三方物件不保證 `thread-safe`。

內建 `ExcelRenderer` 本身偏向無狀態，通常可使用 `Singleton`。若自訂 `renderer` 的 `thread-safe` 或重用行為不明確，建議先從 `Transient` 或 `Scoped` 開始，再依實際需求調整。

第一個匯出範例

```
// 建立文件
SpreadsheetDocument document = SpreadsheetManager.CreateDocument();

// 建立工作表
SheetDefinition sheet = document.CreateSheet("工作表1");

// 建立 GridTemplate 並加入內容
GridTemplate template = new GridTemplate();
template.CreateRow();
template.CreateCell("Hello, SpreadsheetExporter!");
sheet.AddTemplate(template);

// 匯出至檔案
document.ExportFile($"C:\\Sample{document.FileNameExtension}");
```

下一步

- [SpreadsheetDocument](#) - 了解活頁簿層級的設定與匯出
- [SheetDefinition](#) - 理解工作表管理
- [Templates](#) - 探索不同的資料結構化方式
- [自訂樣式](#) - 自訂儲存格樣式與格式

SpreadsheetDocument

`SpreadsheetDocument` 代表一份試算表文件，負責管理工作表定義並協調最終渲染輸出。

建立文件

若是非 DI 專案，建議使用 `SpreadsheetManager` 作為啟動入口：

透過 `SpreadsheetManager.CreateDocument()` 建立文件：

```
SpreadsheetManager.SetRenderer(static () => new ExcelRenderer());

SpreadsheetDocument document = SpreadsheetManager.CreateDocument();
```

若你的專案已使用 DI，也可直接傳入 `renderer` 實例建立：

```
SpreadsheetDocument document = new SpreadsheetDocument(new ExcelRenderer());
```

例如在 DI 組合根解析 `renderer` 後建立：

```
ISpreadsheetRenderer renderer = serviceProvider.GetRequiredService<ISpreadsheetRenderer>();
SpreadsheetDocument document = new SpreadsheetDocument(renderer);
```

文件層級設定

屬性	說明
<code>DefaultFont</code>	套用至整份文件的預設字型； <code>null</code> 代表使用 <code>renderer</code> 預設值
<code>DefaultSheetName</code>	自動產生工作表名稱時使用的前綴（預設值：「工作表」）
<code>Styles</code>	文件層級 named styles，供 JSON template 以 style name 引用
<code>ContentType</code>	匯出格式的 MIME 類型，來自 <code>renderer</code>
<code>FileNameExtension</code>	匯出格式的副檔名（含前置點），來自 <code>renderer</code>

```
SpreadsheetDocument document = SpreadsheetManager.CreateDocument();
document.DefaultFont = new CellFont("Calibri", 11);
document.DefaultSheetName = "Sheet";
```

文件層級 named styles 可透過 `UseStyles(...)` 設定：

```
SpreadsheetStyleRegistry styles = new();
styles.Set("Header", new CellStyle(
    HasBorder: true,
    Font: new CellFont("Calibri", 12, Style: FontStyles.Bold)
));

document.UseStyles(styles);
```

工作表管理

建立工作表

```
// 建立指定名稱的工作表
SheetDefinition sheet = document.CreateSheet("銷售報表");

// 建立指定名稱與預設列高的工作表
SheetDefinition sheet = document.CreateSheet("銷售報表", defaultRowHeight: 20);

// 自動命名 (依序產生「工作表1」、「工作表2」等)
SheetDefinition sheet = document.CreateSheet();
```

若傳入重複名稱，會自動加上序號後綴（例如 銷售報表(1)）。

存取工作表

```
// 取得最後一個工作表 (若無則自動建立)
SheetDefinition last = document.LastSheet;

// 依零基索引取得
SheetDefinition first = document.GetSheet(0);

// 安全取得
if (document.TryGetSheet(0, out SheetDefinition sheet)) {
    // ...
}
```

版面診斷

`GetLayoutDiagnostics()` 可在匯出前檢查工作表版面。此方法只回傳診斷結果，不會輸出檔案，也不會改變文件內容。

```

IReadOnlyList<LayoutDiagnostic> diagnostics = document.GetLayoutDiagnostics();

foreach (LayoutDiagnostic diagnostic in diagnostics) {
    Console.WriteLine($"{diagnostic.Code}: {diagnostic.Message}");
}

```

若希望在發現診斷結果時直接中止流程，可使用 `ValidateLayout()`：

```

try {
    document.ValidateLayout();
    document.ExportFile($"output{document.FileNameExtension}");
} catch (SpreadsheetLayoutException ex) {
    foreach (LayoutDiagnostic diagnostic in ex.Diagnostics) {
        Console.WriteLine($"{diagnostic.Code}: {diagnostic.Message}");
    }
}

```

目前診斷範圍包含：

- 儲存格 range 座標或大小是否有效。
- 儲存格 range 是否重疊。
- 同一儲存格是否同時定義 value 與 formula。
- 欄寬與列高是否使用有效值。

`Export()` 不會自動呼叫 `ValidateLayout()`，因此既有匯出流程維持原本行為。需要 render 前驗證時，由呼叫端明確呼叫診斷 API。

匯出

匯出至位元組陣列

適用於網頁回應或記憶體內處理：

```

public IActionResult Download() {
    SpreadsheetDocument document = SpreadsheetManager.CreateDocument();
    SheetDefinition sheet = document.CreateSheet("報表");
    sheet.AddTemplate(BuildTemplate());

    return File(
        document.Export(),
        document.ContentType,
        $"report{document.FileNameExtension}"
    );
}

```

```
);  
}
```

匯出至檔案

```
// 若檔案存在則覆寫 (預設)  
document.ExportFile($"C:\\Reports\\output{document.FileNameExtension}");  
  
// 若檔案存在則拋出 IOException  
document.ExportFile(  
    $"C:\\Reports\\output{document.FileNameExtension}",  
    SpreadsheetFileMode.CreateNew  
);
```

SpreadsheetFileMode 選項:

- `Create` - 建立新檔案，若存在則覆寫 (預設)
- `CreateNew` - 建立新檔案，若存在則拋出例外

從 JSON 建立文件

`SpreadsheetDocument.FromJson` 可從 JSON 字串直接建立完整文件，適合將報表結構外部化配置：

```
string json = File.ReadAllText("report-template.json");  
SpreadsheetDocument document = SpreadsheetDocument.FromJson(json);  
document.ExportFile($"output{document.FileNameExtension}");
```

若要讓 JSON template 引用程式碼中建立的文件層級 named styles，可傳入 `SpreadsheetStyleRegistry`：

```
SpreadsheetStyleRegistry styles = new();  
styles.Set("Header", new CellStyle(  
    HasBorder: true,  
    Font: new CellFont("Calibri", 12, Style: FontStyles.Bold)  
));  
  
SpreadsheetDocument document = SpreadsheetDocument.FromJson(json, styles);
```

JSON 格式為工作表陣列，目前支援 `Grid`、`RecordSet`、`DataTable` 與 `Merged` 四種 template 類型：

JSON Schema 可參考 [spreadsheet-exporter.schema.json](#)。因為 JSON root 是工作表陣列，不建議把 `$schema` 寫入同一份 template；建議在編輯器設定中將該 schema 綁定到 template 檔案。

```
[
  {
    "SheetName": "Report",
    "DefaultRowHeight": 20,
    "ColumnWidths": [
      { "Index": 0, "Width": 18 },
      { "Index": 1, "Width": 24 }
    ],
    "Templates": [
      {
        "Type": "Grid",
        "Rows": [
          {
            "Height": 24,
            "Cells": [
              { "Value": "標題", "ColumnSpan": 2, "Style": { "HorizontalAlignment": "Center"

```

工作表層級支援欄位

欄位	型別	說明
SheetName	string	工作表名稱；省略時會自動命名
DefaultRowHeight	number	工作表預設列高
Password	string	工作表保護密碼
FreezePanels	object	凍結窗格設定，需同時提供 Row 與 Column
IsAutoFilterEnabled	boolean	啟用自動篩選
ColumnWidths	object / array	欄寬設定，可用索引物件或 { Index, Width } 陣列
PageSettings	object	列印設定，目前支援 PageOrientation 與 PaperSize
Metadata	object	附加自訂資料，供 renderer 或擴充邏輯讀取

欄位	型別	說明
Styles	object	工作表層級 named styles, 供 template 以 style name 引用
Templates	array	工作表上的 template 定義

FreezePanes 範例:

```
{
  "FreezePanes": {
    "Row": 1,
    "Column": 0
  }
}
```

ColumnWidths 可用兩種格式:

```
{
  "ColumnWidths": {
    "0": 18,
    "1": 24
  }
}
```

```
{
  "ColumnWidths": [
    { "Index": 0, "Width": 18 },
    { "Index": 1, "Width": 24 }
  ]
}
```

Grid template 支援欄位

欄位	型別	說明
Type	string	固定為 Grid
Rows	array	列定義
Rows[].Height	number	該列列高
Rows[].Cells	array	儲存格定義

欄位	型別	說明
Rows[].Cells[].Value	any	固定值
Rows[].Cells[].Formula	string	公式字串
Rows[].Cells[].ColumnSpan	number	欄合併數，預設 1
Rows[].Cells[].RowSpan	number	列合併數，預設 1
Rows[].Cells[].StyleName	string	named style 名稱
Rows[].Cells[].Style	object	儲存格樣式
Rows[].Cells[].DataValidation	object	儲存格資料驗證

Grid 的 Value 與 Formula 只能擇一設定。

RecordSet template 支援欄位

欄位	型別	說明
Type	string	固定為 RecordSet
HeaderHeight	number	標題列高度
RecordHeight	number	資料列高度
Columns	array	欄位定義
Records	array	資料列；每筆記錄會轉成 Dictionary<string, object?>

Columns[] 支援欄位：

欄位	型別	說明
HeaderText	string	欄位標題
HeaderStyleName	string	標題 named style 名稱
HeaderStyle	object	標題樣式
FieldStyleName	string	資料儲存格 named style 名稱
FieldStyle	object	資料儲存格樣式

欄位	型別	說明
FieldKey	string	從 <code>Records[]</code> 取值的欄位名稱
Value	any	固定值
Formula	string	固定公式
DataValidation	object	資料儲存格驗證
Children	array	子欄位，建立多層標題

`RecordSet` 欄位的 `FieldKey`、`Value`、`Formula` 只能擇一設定。

`FieldKey` 支援巢狀物件路徑，例如：

```
{
  "HeaderText": "City",
  "FieldKey": "Customer.Address.City"
}
```

DataTable template 支援欄位

欄位	型別	說明
Type	string	固定為 <code>DataTable</code>
HeaderHeight	number	標題列高度
RecordHeight	number	資料列高度
Columns	array	欄位定義；省略時會依 <code>Records</code> 的物件鍵值推斷欄位
Records	array	資料列；每筆記錄必須是 JSON object

`Columns[]` 支援欄位：

欄位	型別	說明
ColumnName	string	對應 <code>Records[]</code> 的欄位名稱
HeaderText	string / null	欄位標題；省略時使用 <code>ColumnName</code>
HeaderStyleName	string	標題 named style 名稱

欄位	型別	說明
HeaderStyle	object	標題樣式
FieldStyleName	string	資料儲存格 named style 名稱
FieldStyle	object	資料儲存格樣式
Value	any	固定值
Formula	string / null	固定公式
DataValidation	object	資料儲存格驗證

DataTable 欄位的 Value 與 Formula 只能擇一設定。

當 Columns 省略時，欄位順序依 Records 內第一次出現的鍵值順序決定。記錄缺少欄位時，該儲存格值為 null。

Merged template 支援欄位

欄位	型別	說明
Type	string	固定為 Merged
Templates	array	子 template 定義，支援所有已註冊的 JSON template 型別

Merged 會依子 template 順序垂直堆疊版面，行為與程式碼中的 MergedTemplate 一致。

DataValidation 支援欄位

Grid 的 Rows[].Cells[].DataValidation、RecordSet 的 Columns[].DataValidation 與 DataTable 的 Columns[].DataValidation 使用相同欄位：

欄位	型別	說明
ValidationType	string / number	驗證類型，可用 List、Integer、Decimal、Date、Time、TextLength、Custom
Operator	string / number / null	比較運算子，可用 Between、NotBetween、Equal、NotEqual、GreaterThan、LessThan、GreaterThanOrEqual、LessThanOrEqual
Value1	any	第一個比較值

欄位	型別	說明
Value2	any	第二個比較值
ListItems	array / null	清單驗證的可選值，項目必須是字串
Formula	string / null	自訂驗證公式，或數值 / 日期 / 時間驗證的公式來源
IsDropDownShown	boolean	清單驗證是否顯示下拉選單
IsBlankAllowed	boolean	是否允許空白值
ErrorTitle	string / null	驗證錯誤標題
ErrorMessage	string / null	驗證錯誤訊息
IsErrorAlertShown	boolean	是否顯示錯誤提示
PromptTitle	string / null	輸入提示標題
PromptMessage	string / null	輸入提示訊息
IsInputPromptShown	boolean	選取儲存格時是否顯示輸入提示

未設定 `ValidationType` 時，使用 `DataValidation` 的預設驗證類型 `List`。

Style 與 Font 支援欄位

`Styles` 可在文件與工作表層級宣告 named styles。文件層級 styles 透過 `SpreadsheetDocument.FromJson(json, styles)` 或 `document.UseStyles(styles)` 提供；工作表層級 styles 則在 JSON 的 `Styles` 欄位宣告。

template 使用 `StyleName`、`HeaderStyleName` 或 `FieldStyleName` 引用時，會先找工作表層級 style，再找文件層級 style。找到 named style 後，inline style 欄位只覆寫 JSON 中有明確宣告的屬性。例如 inline `Font.Style` 只會覆寫字型樣式，不會清除 named style 中的字型名稱、大小或顏色。

```
{
  "Styles": {
    "Header": {
      "HasBorder": true,
      "Font": { "Style": "Bold" }
    }
  },
  "Templates": [
    {
```

```

    "Type": "Grid",
    "Rows": [
      {
        "Cells": [
          {
            "Value": "Title",
            "StyleName": "Header",
            "Style": {
              "HorizontalAlignment": "Center"
            }
          }
        ]
      }
    ]
  }
]
}

```

Grid 的 Style、RecordSet 的 HeaderStyle 與 FieldStyle，以及 DataTable 的 HeaderStyle 與 FieldStyle 目前支援以下欄位：

欄位	型別	說明
HorizontalAlignment	string / number	水平對齊
VerticalAlignment	string / number	垂直對齊
HasBorder	boolean	是否套用框線
WrapText	boolean	是否自動換行
BackgroundColor	string / object	背景色
DataFormat	string / null	Excel 格式字串
IsLocked	boolean	是否鎖定儲存格
Font	object	字型設定

Font 支援欄位：

欄位	型別	說明
Name	string / null	字型名稱

欄位	型別	說明
Size	number	字型大小
Color	string / object	字型顏色
Style	string / number / array	字型樣式，可用 Bold 、 <i>Italic</i> 、 <u>Underline</u> 、 Strikeout 或其數值

色彩格式支援：

- 命名色彩，例如 `Red`
- HTML 色碼，例如 `#FFAA00`
- ARGB / RGB 物件，例如 `{ "R": 255, "G": 170, "B": 0 }`

列舉值支援名稱或數值，且字串比對不分大小寫。

完整範例

```
[
  {
    "SheetName": "Orders",
    "DefaultRowHeight": 20,
    "Password": "sheet-pass",
    "FreezePanes": { "Row": 1, "Column": 0 },
    "IsAutoFilterEnabled": true,
    "ColumnWidths": {
      "0": 12,
      "1": 24,
      "2": 18
    },
    "PageSettings": {
      "PageOrientation": "Landscape",
      "PaperSize": "A4"
    },
    "Metadata": {
      "reportType": "monthly",
      "generatedBy": "system"
    },
    "Templates": [
      {
        "Type": "Grid",
        "Rows": [
          {
            "Height": 28,
            "Cells": [
```

```

    {
      "Value": "Order Summary",
      "ColumnSpan": 3,
      "Style": {
        "HorizontalAlignment": "Center",
        "HasBorder": true,
        "BackgroundColor": "#1F4E79",
        "Font": {
          "Name": "Calibri",
          "Size": 14,
          "Color": "White",
          "Style": ["Bold"]
        }
      }
    }
  ]
}
],
{
  "Type": "RecordSet",
  "HeaderHeight": 22,
  "RecordHeight": 20,
  "Columns": [
    { "HeaderText": "Order ID", "FieldKey": "OrderId" },
    {
      "HeaderText": "Customer",
      "FieldKey": "Customer.Name"
    },
    {
      "HeaderText": "Amount",
      "FieldKey": "Amount",
      "FieldStyle": {
        "HorizontalAlignment": "Right",
        "DataFormat": "#,##0.00"
      },
      "DataValidation": {
        "ValidationType": "Decimal",
        "Operator": "GreaterThanOrEqual",
        "Value1": 0,
        "ErrorTitle": "Invalid amount",
        "ErrorMessage": "Amount must be greater than or equal to zero."
      }
    }
  ]
},
"Records": [

```



```

{
  "OrderId": 1001,
  "Customer": {
    "Name": "Northwind"
  },
  "Amount": 1250.40
},
{
  "OrderId": 1002,
  "Customer": {
    "Name": "Adventure Works"
  },
  "Amount": 980.00
}
]
}
]
}
]

```

目前未支援的 JSON 功能

- 需要以程式碼動態計算的 generator 邏輯。
- 自訂 template 型別，除非先自行註冊對應的 JSON parser。

JSON 錯誤診斷

`SpreadsheetDocument.FromJson(...)` 解析失敗時，內建 parser 會在錯誤訊息中附上診斷代碼與 JSON path。JSON root 是工作表陣列，因此第一張工作表的路徑會從 `$(0)` 開始。

SE-JSON-001: `$(0).Templates[0].Rows[0].Cells[0].ColumnSpan` must be a 32-bit integer.

SE-JSON-002: `$(0).Templates[0].Type` is required.

SE-JSON-003: `$(0).Templates[0].Rows[0].Cells[0]` cannot specify both 'Value' and 'Formula'.

註冊自訂 JSON parser

`SpreadsheetDocument.FromJson(...)` 會透過 `JsonTemplateRegistry` 依 `Type` 欄位尋找對應的 parser。

內建已註冊的型別有：

- `DataTable`
- `Grid`
- `Merged`
- `RecordSet`

若要支援自訂 template, 可實作 `ITemplateJsonParser`, 再於呼叫 `FromJson(...)` 前註冊:

```
using System.Text.Json;
using CloudyWing.SpreadsheetExporter;
using CloudyWing.SpreadsheetExporter.Templates;
using CloudyWing.SpreadsheetExporter.Templates.Json;

public class ReportHeaderTemplateJsonParser : ITemplateJsonParser {
    public string TypeName => "ReportHeader";

    public ISheetTemplate Parse(JsonElement element) {
        string title = element.GetProperty("Title").GetString(!);
        int columnSpan = element.GetProperty("ColumnSpan").GetInt32();
        return new ReportHeaderTemplate(title, columnSpan);
    }
}

JsonTemplateRegistry.Register(new ReportHeaderTemplateJsonParser());

SpreadsheetManager.SetRenderer(static () => new ExcelRenderer());
SpreadsheetDocument document = SpreadsheetDocument.FromJson(json);
```

對應 JSON:

```
{
  "Type": "ReportHeader",
  "Title": "Q1 Sales Report",
  "ColumnSpan": 4
}
```

若需要完全重設註冊內容, 可先呼叫 `JsonTemplateRegistry.Clear()`, 再視需要呼叫 `JsonTemplateRegistry.RegisterBuiltins()` 與自訂 parser 註冊。

自訂 renderer 行為

`ExcelRenderer` 提供 `OnSheetCreated` 虛擬方法, 可在工作表建立後透過 `ClosedXML` API 進行額外設定:

```
using ClosedXML.Excel;
using CloudyWing.SpreadsheetExporter;
using CloudyWing.SpreadsheetExporter.Renderer.ClosedXML;

public class CustomExcelRenderer : ExcelRenderer {
    protected override void OnSheetCreated(IXLWorksheet worksheet, SheetLayout context) {
        // 透過 ClosedXML API 進行自訂設定
    }
}
```

```

        worksheet.SheetView.ShowGridLines = false;
    }
}

SpreadsheetManager.SetRenderer(static () => new CustomExcelRenderer());

```

Renderer 能力宣告

Renderer 可選擇實作 `ISpreadsheetRendererWithCapabilities`，宣告自身支援的功能。既有只實作 `ISpreadsheetRenderer` 的 renderer 仍可正常使用。

```

ISpreadsheetRenderer renderer = new ExcelRenderer();

if (renderer is ISpreadsheetRendererWithCapabilities capableRenderer) {
    RendererCapabilities capabilities = capableRenderer.Capabilities;

    if (capabilities.SupportsMergedCells) {
        // 可使用合併儲存格相關版面。
    }
}

```

`SpreadsheetDocument.GetRendererCapabilityDiagnostics()` 可檢查文件是否使用 renderer 未宣告支援的能力。此方法只在 renderer 實作 `ISpreadsheetRendererWithCapabilities` 時回傳診斷；既有 renderer 未宣告能力時，診斷結果為空集合。

```

IReadOnlyList<LayoutDiagnostic> diagnostics = document.GetRendererCapabilityDiagnostics();

foreach (LayoutDiagnostic diagnostic in diagnostics) {
    Console.WriteLine($"{diagnostic.Code}: {diagnostic.Message}");
}

```

相關主題

- [入門指南](#) - 從零開始設定 SpreadsheetExporter
- [SheetDefinition](#) - 工作表管理與設定
- [Templates](#) - 探索不同的資料結構化方式
- [Migration Notes](#) - 檢視不同版本之間的 JSON 與 API 調整
- [自訂樣式](#) - 透過 SpreadsheetManager 設定全域儲存格樣式

SheetDefinition

`SheetDefinition` 代表文件中的單一工作表，包含 templates、欄寬設定、頁面設定與保護密碼。

基本設定

工作表名稱

`CreateSheet` 傳入的名稱即為工作表顯示名稱，建立後仍可修改：

```
SheetDefinition sheet = document.CreateSheet("銷售報表");  
sheet.SheetName = "Q1 銷售報表";
```

預設列高

```
SheetDefinition sheet = document.CreateSheet("報表", defaultRowHeight: 20);  
// 或建立後設定  
sheet.DefaultRowHeight = 20;
```

工作表保護

為工作表設定密碼保護（保護後使用者無法編輯未解鎖的儲存格）：

```
SheetDefinition sheet = document.CreateSheet("報表");  
sheet.Password = "your-password";
```

欄位設定

使用 `SetColumnWidth` 設定欄寬（索引從 0 開始）。方法支援 Fluent 鏈式呼叫：

```
sheet  
    .SetColumnWidth(0, 18d) // 設定指定寬度（字元單位）  
    .SetColumnWidth(1, Constants.HiddenColumn) // 隱藏欄位  
    .SetColumnWidth(2, Constants.AutoFitColumnWidth); // 自動調整欄寬
```

頁面設定

透過 `PageSettings` 設定列印相關選項：

```
sheet.PageSettings.PageOrientation = PageOrientation.Landscape;  
sheet.PageSettings.PaperSize = PaperSize.A4;
```

PageOrientation 選項：

- **Portrait** - 直向 (預設)
- **Landscape** - 橫向

使用 Templates

Templates 會依加入順序**垂直堆疊**。若 **template1** 佔用 3 列，**template2** 將從第 4 列開始。

```
// 加入單一 template
sheet.AddTemplate(template1);

// 加入多個 templates (params 語法)
sheet.AddTemplates(template2, template3);

// 加入集合
sheet.AddTemplates(templateList);

// Fluent 鏈式呼叫
sheet
    .AddTemplate(headerTemplate)
    .AddTemplates(dataTemplate, footerTemplate);
```

NOTE

同一份文件可建立多個工作表，每個工作表各自維護獨立的 templates 與設定。

Metadata

Metadata 字典可附加任意資訊，供自訂 renderer 使用（例如在 **OnSheetCreated** 中讀取）：

```
sheet.Metadata["reportType"] = "quarterly";
sheet.Metadata["generatedBy"] = "system";
```

在自訂 renderer 中讀取：

```
using ClosedXML.Excel;
using CloudyWing.SpreadsheetExporter;
using CloudyWing.SpreadsheetExporter.Renderer.ClosedXML;

public class CustomExcelRenderer : ExcelRenderer {
    protected override void OnSheetCreated(IXLWorksheet worksheet, SheetLayout context) {
```

```

        if (context.Metadata.TryGetValue("reportType", out object reportType)) {
            worksheet.Tab.Color = reportType as string == "quarterly"
                ? XLColor.Blue
                : XLColor.Gray;
        }
    }
}

```

用擴充方法封裝慣用設定

若某些工作表設定在專案內會重複出現，可以用擴充方法包起來，而不是讓呼叫端直接操作 `Metadata` 鍵名或重複設定流程。

```

public static class SheetDefinitionExtensions {
    public static SheetDefinition SetReportType(this SheetDefinition sheet, string
reportType) {
        sheet.Metadata["reportType"] = reportType;
        return sheet;
    }

    public static SheetDefinition ConfigureQuarterlyReport(this SheetDefinition sheet) {
        return sheet
            .SetReportType("quarterly")
            .SetFreezePanels(0, 1)
            .SetAutoFilterEnabled();
    }
}

```

使用時可保持呼叫端簡潔：

```

document
    .CreateSheet("Q1")
    .ConfigureQuarterlyReport()
    .AddTemplate(template);

```

這類擴充方法適合包裝：

- 專案內固定會重複的 `Metadata` 鍵值。
- 常見的工作表初始化流程。
- 特定 `renderer` 需要的慣例設定。

相關主題

- [入門指南](#) - 了解 SheetDefinition 在整體匯出流程中的角色
- [SpreadsheetDocument](#) - 學習如何透過文件建立工作表
- [Templates](#) - 探索可加入工作表的各種範本類型
- [自訂樣式](#) - 設定工作表中儲存格的預設樣式

Templates

Templates 定義工作表儲存格的結構與內容。SpreadsheetExporter 提供四種內建 template 類型，並支援自訂擴充。

本文內容

- [GridTemplate](#) - 手動逐格配置
- [DataTableTemplate](#) - 基於 DataTable 的匯出
- [RecordSetTemplate](#) - 強型別集合
- [MergedTemplate](#) - 合併多個 templates
- [自訂 Templates](#) - 建立可重複使用的自訂 template

若你是透過 `SpreadsheetDocument.FromJson(...)` 建立文件，目前 JSON 支援 `Grid`、`RecordSet`、`DataTable` 與 `Merged` 四種 template 型別。完整 JSON 欄位請參考 [SpreadsheetDocument](#) 的 JSON 章節。

GridTemplate

`GridTemplate` 提供精細的儲存格配置控制，類似 HTML 的 `<table>`、`<tr>` 與 `<td>`。支援方法鏈（Method Chaining）。

建立列

```
GridTemplate template = new GridTemplate();

// 預設列高
template.CreateRow();

// 指定列高
template.CreateRow(20d);

// 隱藏列
template.CreateRow(Constants.HiddenRow);

// 自動調整列高
template.CreateRow(Constants.AutoFitRowHeight);
```

建立儲存格

```
GridTemplate template = new GridTemplate();
template.CreateRow();

// 簡單值儲存格
template.CreateCell("Hello");
```



```
// 合併儲存格 (columnSpan=2, rowSpan=2)
template.CreateCell("Merged", columnSpan: 2, rowSpan: 2);

// 自訂樣式
CellStyle style = new CellStyle(HorizontalAlignment: HorizontalAlignment.Center,
HasBorder: true);
template.CreateCell("Styled", cellStyle: style);
```

公式儲存格

```
// 傳入 (cellIndex, rowIndex) 的 lambda
template.CreateRow()
    .CreateCell((cellIndex, rowIndex) => $"=SUM(A{rowIndex + 1},1)");
```

完整自訂儲存格

使用 `Action<Cell>` 多載可同時設定值、公式與資料驗證：

```
template.CreateRow();
template.CreateCell(cell => {
    cell.ValueGenerator = (_, _) => "請選擇";
    cell.DataValidationGenerator = (_, _) => new DataValidation {
        ValidationType = DataValidationType.List,
        ListItems = ["選項 A", "選項 B", "選項 C"],
        IsDropDownShown = true
    };
});
```

DataTableTemplate

直接將 `System.Data.DataTable` 匯出至試算表，自動對應欄位名稱為標題列。

```
System.Data.DataTable dataTable = new System.Data.DataTable();
dataTable.Columns.Add("Name", typeof(string));
dataTable.Columns.Add("Age", typeof(int));
dataTable.Rows.Add("John", 30);
dataTable.Rows.Add("Mary", 25);

DataTableTemplate template = new DataTableTemplate(dataTable) {
    HeaderHeight = 25,
    RecordHeight = 20
};
```

```

/*
輸出:
| ---- | --- |
| Name | Age |
| ---- | --- |
| John | 30 |
| ---- | --- |
| Mary | 25 |
| ---- | --- |
*/

```

資料驗證

`DataTableColumn.FieldDataValidationGenerator` 可依資料值回傳驗證規則：

```

DataTableTemplate template = new DataTableTemplate(dataTable);
template.Columns[1].FieldDataValidationGenerator = value => new DataValidation {
    ValidationType = DataValidationType.Integer,
    Operator = DataValidationOperator.Between,
    Value1 = 1,
    Value2 = 120
};

```

RecordSetTemplate

適用於強型別資料集合，支援欄位綁定、資料轉換、條件樣式、多層標題與資料驗證。

基本用法

```

public class Order {
    public int Id { get; set; }
    public string Customer { get; set; }
    public decimal Amount { get; set; }
}

List<Order> orders = [
    new Order { Id = 1, Customer = "Northwind", Amount = 1250.40m },
    new Order { Id = 2, Customer = "Adventure Works", Amount = 980.00m }
];

RecordSetTemplate<Order> template = new RecordSetTemplate<Order>(orders);
template.Columns.Add<int>("編號", order => order.Id);

```

```
template.Columns.Add<string>("客戶", order => order.Customer);
template.Columns.Add<decimal>("金額", order => order.Amount);
```

資料轉換

使用 `configureGenerators` 參數自訂值或公式產生方式：

```
// 轉換值
template.Columns.Add<string>(
    "大寫客戶",
    order => order.Customer,
    configureGenerators: config => config.UseValue(ctx => ctx.Value.ToUpper())
);

// 不綁定欄位，完全自訂值
template.Columns.Add(
    "合併資料",
    configureGenerators: config => config.UseValue(
        ctx => $"{ctx.Record.Id} - {ctx.Record.Customer}"
    )
);
```

公式

```
// context.RowIndex 為資料列的零基索引（標題列不計入）
template.Columns.Add(
    "含稅金額",
    configureGenerators: config => config.UseFormula(
        context => $"C{context.RowIndex + 2}*1.05"
    )
);
```

條件樣式

```
CellStyle currencyStyle = new CellStyle(
    HorizontalAlignment: HorizontalAlignment.Right,
    DataFormat: "#,##0.00"
);

template.Columns.Add<decimal>(
    "金額",
    order => order.Amount,
    fieldStyleGenerator: ctx => ctx.Value >= 1000
        ? currencyStyle with { BackgroundColor = Color.LightGreen }
    )
```

```
        : currencyStyle
    );
```

多層標題

```
template.Columns
    .Add<int>("編號", order => order.Id)
    .Add("客戶資訊")
    .GetLastColumn().ChildColumns
        .Add("客戶", order => order.Customer)
        .Add("地區", order => order.Region);
```

```
/*
輸出標題:
| ---- | ----- |
|      | 客戶資訊 |
| 編號 | 客戶 | 地區 |
*/
```

搭配工作表設定凍結窗格與自動篩選

```
RecordSetTemplate<Order> template = new RecordSetTemplate<Order>(orders);
```

```
SheetDefinition sheet = document.CreateSheet("Orders");
```

```
sheet
    .AddTemplate(template)
    .SetFreezePanels(0, 1)
    .SetAutoFilterEnabled();
```

資料驗證

```
template.Columns.Add<string>(
    "地區",
    order => order.Region,
    configureGenerators: config => config.UseDataValidation(_ => new DataValidation {
        ValidationType = DataValidationType.List,
        ListItems = ["North", "South", "East", "West"],
        ErrorTitle = "無效地區",
        ErrorMessage = "請選擇有效的銷售地區。",
        IsErrorAlertShown = true
    })
);
```

MergedTemplate

合併多個 templates，垂直堆疊輸出。適合將標題、資料表與頁尾組合成單一 template 傳入工作表。

```
MergedTemplate merged = new MergedTemplate(headerTemplate, dataTemplate, footerTemplate);
sheet.AddTemplate(merged);
```

自訂 Templates

實作 `ISheetTemplate` 介面可建立可重複使用的自訂 template：

```
public class ReportHeaderTemplate : ISheetTemplate {
    private readonly GridTemplate gridTemplate = new GridTemplate();

    public ReportHeaderTemplate(string title, int columnSpan) {
        CellStyle titleStyle = SpreadsheetManager.DefaultCellStyles.GridCellStyle with {
            HorizontalAlignment = HorizontalAlignment.Center,
            Font = SpreadsheetManager.DefaultCellStyles.GridCellStyle.Font with { Size =
14 }
        };

        gridTemplate.CreateRow(28);
        gridTemplate.CreateCell(title, columnSpan: columnSpan, cellStyle: titleStyle);

        gridTemplate.CreateRow();
        gridTemplate.CreateCell(
            $"產生時間: {DateTimeOffset.Now:yyyy-MM-dd HH:mm:ss zzz}",
            columnSpan: columnSpan
        );
    }

    public int ColumnSpan => gridTemplate.ColumnSpan;

    public int RowSpan => gridTemplate.RowSpan;

    public TemplateLayout GetLayout() => gridTemplate.GetLayout();
}
```

使用方式：

```
sheet.AddTemplate(new ReportHeaderTemplate("Q1 銷售報表", columnSpan: 4));
sheet.AddTemplate(dataTemplate);
```

相關主題

- [入門指南](#) - 了解如何在專案中開始使用 templates
- [SpreadsheetDocument](#) - 了解 template 與文件的整體協作流程
- [SheetDefinition](#) - 了解如何將 templates 加入工作表
- [自訂樣式](#) - 為 template 中的儲存格套用自訂樣式

自訂樣式

`SpreadsheetManager.DefaultCellStyles` 提供全域的預設儲存格樣式，各 `template` 類型會自動套用對應的預設值。

預設樣式

`CellStyleConfiguration` 包含以下四個樣式屬性：

屬性	用途
<code>CellStyle</code>	一般儲存格的基礎樣式
<code>GridCellStyle</code>	<code>GridTemplate</code> 儲存格使用的預設樣式
<code>HeaderStyle</code>	<code>RecordSetTemplate</code> 標題列使用的預設樣式
<code>FieldStyle</code>	<code>RecordSetTemplate</code> 資料列使用的預設樣式

更改預設樣式

`CellStyleConfiguration` 使用 `init` 屬性，建立時透過物件初始化器覆寫需要變更的樣式，其餘保留建構子預設值：

基於現有樣式修改

```
CellStyleConfiguration existing = SpreadsheetManager.DefaultCellStyles;
```

```
SpreadsheetManager.DefaultCellStyles = new CellStyleConfiguration {
    CellStyle = existing.CellStyle with {
        Font = existing.CellStyle.Font with { Size = 12 }
    },
    GridCellStyle = existing.GridCellStyle with {
        Font = existing.GridCellStyle.Font with { Size = 12 }
    },
    HeaderStyle = existing.HeaderStyle with {
        Font = existing.HeaderStyle.Font with { Size = 12 }
    },
    FieldStyle = existing.FieldStyle with {
        Font = existing.FieldStyle.Font with { Size = 12 }
    }
};
```

完整自訂

```

CellFont customFont = new CellFont(
    Name: "微軟正黑體",
    Size: 12,
    Color: Color.Black,
    Style: FontStyles.None
);

CellStyle baseStyle = new CellStyle(
    HorizontalAlignment: HorizontalAlignment.Left,
    VerticalAlignment: VerticalAlignment.Middle,
    Font: customFont
);

SpreadsheetManager.DefaultCellStyles = new CellStyleConfiguration {
    CellStyle = baseStyle,
    GridCellStyle = baseStyle,
    HeaderStyle = baseStyle with {
        HorizontalAlignment = HorizontalAlignment.Center,
        HasBorder = true,
        Font = customFont with { Style = FontStyles.Bold }
    },
    FieldStyle = baseStyle with {
        HasBorder = true
    }
};

```

從設定檔讀取樣式

設定檔結構

在專案中建立 `spreadsheet.json`，並設定「複製到輸出目錄」：

```

{
  "Spreadsheet": {
    "FontName": "新細明體",
    "FontSize": 10,
    "HorizontalAlignment": "Center",
    "VerticalAlignment": "Middle"
  }
}

```

載入設定


```

IConfigurationRoot config = new ConfigurationBuilder()
    .SetBasePath(AppDomain.CurrentDomain.BaseDirectory)
    .AddJsonFile("spreadsheet.json", optional: false, reloadOnChange: true)
    .Build();

void ApplyStyles() {
    IConfigurationSection section = config.GetSection("Spreadsheet");

    CellFont font = new CellFont(
        Name: section["FontName"],
        Size: short.Parse(section["FontSize"]!)
    );

    CellStyle baseStyle = new CellStyle(
        HorizontalAlignment: Enum.Parse<HorizontalAlignment>
(section["HorizontalAlignment"]!),
        VerticalAlignment: Enum.Parse<VerticalAlignment>(section["VerticalAlignment"]!),
        Font: font
    );

    SpreadsheetManager.DefaultCellStyles = new CellStyleConfiguration {
        CellStyle = baseStyle,
        GridCellStyle = baseStyle,
        HeaderStyle = baseStyle with {
            HasBorder = true,
            Font = font with { Style = FontStyles.Bold }
        },
        FieldStyle = baseStyle with { HasBorder = true }
    };
}

ChangeToken.OnChange(() => config.GetReloadToken(), ApplyStyles);
ApplyStyles();

```

TIP

使用 `ChangeToken.OnChange` 可在設定檔變更時自動重新載入樣式，無需重新啟動應用程式。

在 Template 層級覆寫樣式

各 template 也可以在加入欄位時直接傳入樣式，優先於全域預設值：

```

CellStyle customHeader = SpreadsheetManager.DefaultCellStyles.HeaderStyle with {
    BackgroundColor = Color.FromArgb(31, 78, 121),
    Font = SpreadsheetManager.DefaultCellStyles.HeaderStyle.Font with {
        Color = Color.White
    }
};

template.Columns.Add<string>("客戶", order => order.Customer, headerStyle: customHeader);

```

JSON named styles

若報表結構由 JSON 描述，可用 `SpreadsheetStyleRegistry` 宣告可重複引用的 named styles：

```

SpreadsheetStyleRegistry styles = new();
styles.Set("Header", new CellStyle(
    HasBorder: true,
    Font: new CellFont("Calibri", 12, Style: FontStyles.Bold)
));

SpreadsheetDocument document = SpreadsheetDocument.FromJson(json, styles);

```

JSON template 可用 `StyleName`、`HeaderStyleName` 或 `FieldStyleName` 引用 named style。若同時提供 inline style，inline style 只覆寫有明確宣告的屬性：

```

{
  "Type": "Grid",
  "Rows": [
    {
      "Cells": [
        {
          "Value": "Title",
          "StyleName": "Header",
          "Style": {
            "HorizontalAlignment": "Center"
          }
        }
      ]
    }
  ]
}

```

相關主題

- [入門指南](#) - 了解 SpreadsheetManager 的設定時機
- [SpreadsheetDocument](#) - 學習文件層級的字型設定
- [Templates](#) - 學習在 template 中覆寫或使用自訂樣式

Migration Notes

本文整理 `SpreadsheetExporter` 各版本之間的升級重點，方便後續持續追加新的遷移內容。

v2 → v3

核心命名調整

- `Sheetter` 改為 `SheetDefinition`
- `SheetterContext` 改為 `SheetLayout`
- `TemplateContext` 改為 `TemplateLayout`
- `GetContext()` 改為 `GetLayout()`
- `ITemplate` 改為 `ISheetTemplate`
- `ISpreadsheetExporter` / `ExporterBase` 改為 `ISpreadsheetRenderer`

Renderer 與套件調整

v3 將 `renderer` 抽離成 `ISpreadsheetRenderer`，並以 `SpreadsheetExporter.Renderer.ClosedXML` 作為目前的 `.xlsx` 輸出實作。

- 舊的 EPPlus / NPOI solution 專案已移除
- 請改用 `CloudyWing.SpreadsheetExporter.Renderer.ClosedXML`
- 使用入口改為 `ExcelRenderer`

若是非 DI 專案，可透過 `SpreadsheetManager.SetRenderer(...)` 完成初始化：

```
using CloudyWing.SpreadsheetExporter;
using CloudyWing.SpreadsheetExporter.Renderer.ClosedXML;

SpreadsheetManager.SetRenderer(static () => new ExcelRenderer());
SpreadsheetDocument document = SpreadsheetManager.CreateDocument();
```

若是 DI 專案，則可直接註冊 `ISpreadsheetRenderer`，並在使用處建立 `SpreadsheetDocument`：

```
using CloudyWing.SpreadsheetExporter;
using CloudyWing.SpreadsheetExporter.Renderer.ClosedXML;
using Microsoft.Extensions.DependencyInjection;

ServiceCollection services = new();
services.AddSingleton<ISpreadsheetRenderer>(_ => new ExcelRenderer());

ServiceProvider serviceProvider = services.BuildServiceProvider();
ISpreadsheetRenderer renderer = serviceProvider.GetRequiredService<ISpreadsheetRenderer>();
```

```
SpreadsheetDocument document = new(renderer);
```

工作表層級設定調整

`FreezePanels` 與 `IsAutoFilterEnabled` 在 v3 屬於 `SheetDefinition` 的設定，不再掛在 `RecordSetTemplate`。

```
SheetDefinition sheet = document.CreateSheet("Orders");
sheet
    .SetFreezePanels(0, 1)
    .SetAutoFilterEnabled();
```

RecordSet 階層欄位調整

`RecordSetColumnCollection<T>` 移除了 `AddChildToLastColumn(...)` 系列多載，改為先取得最後一個欄位，再對其 `ChildColumns` 操作。

```
template.Columns
    .Add("Order Info")
    .GetLastColumn().ChildColumns
        .Add("Order ID", static x => x.OrderId)
        .Add("Customer", static x => x.Customer);
```

Cell Generator 型別調整

`Cell` 上的 `generator` 屬性型別已從 `Func<int, int, T>` 改為 `CellGenerator<T>`。

多數直接寫 `lambda` 的情境通常不需要改寫，但若有明確宣告舊 `delegate` 型別，請一併更新。

相關文件

- [入門指南](#)
- [SpreadsheetDocument \(文件\)](#)
- [SheetDefinition \(工作表\)](#)
- [Templates \(範本\)](#)